CARE CUBE

Espresso Maker… never mind — Med Tech

Building the Care Cube Circuit in Cirkit Designer (ESP32-S3 Simulator)

Full-replica build + live temperature/humidity streaming to the Care Cube mobile app

Platform: Cirkit Designer — `app.cirkitdesigner.com`
Target board: ESP32-S3 (simulated)
Companion code: `cirkit/sketch.ino`
Repo: `care_cube`
Estimated build time: 15–25 minutes

1. Overview

This guide builds a complete, simulated copy of the Care Cube medicine box inside Cirkit Designer and connects it to the **same MQTT broker** the Care Cube mobile app uses, so real sensor values appear on your phone.

What you will have when finished

- An ESP32-S3 running the Care Cube firmware (simulated).
- DHT22 temperature/humidity readings published every 30 seconds to HiveMQ Cloud topic `carecube/carecube-001/status`.
- Dose reminders at 08:00, 10:00, 12:00, 14:00, 16:00 and 20:00 that turn on an LED + buzzer and publish events on `carecube/carecube-001/event`.
- Schedule changes pushed from the app arriving on `carecube/carecube-001/schedule/set`.
- The temperature and humidity cards on the phone app updating live.

Prerequisites

- A desktop browser (Chrome or Edge recommended) with internet access.
- A free Cirkit Designer account (Sign In at `app.cirkitdesigner.com`).
- The firmware file `cirkit/sketch.ino` from the repository (Chapter 8 of this guide).
- (Optional but recommended) the Care Cube app installed on a phone for the end-to-end verification in Chapter 10.

> **Important:** Cirkit Designer does not support importing a circuit from a file. The workspace is cloud-based and each project lives at `app.cirkitdesigner.com/project/<uuid>`. You build the circuit once, manually, inside the browser using the steps below.

2. Create the project

1. Open `https://app.cirkitdesigner.com` in your browser.
2. Make sure you are signed in (top-right corner: **Sign in** / **Sign up**).
3. Click **Start a new circuit project** (blank project).
4. A new empty workspace opens. The left panel is the **Parts** library, the center is the **canvas**, and the top-right area has the **Design / Code / Simulate / Upload** tabs.

> This simulator supports the **ESP32-S3** (Simulation Features table shows the S3 — not the plain ESP32). Every next step in this guide assumes the ESP32-S3 dev board.

3. Parts to place on the canvas

Search each name in the Parts panel, then drag it onto the canvas. You can rotate a part (selection toolbar) and zoom with `Ctrl + wheel`.

Part	Qty	Search / notes
ESP32-S3 Dev Board	1	Search `ESP32 S3`. This is your main board; everything else wires into it.
DHT22 (temperature & humidity)	1	Search `DHT22`. 4-pin breakout: VCC, DATA, NC?, GND.
LCD 20×4 with I2C	1	Search `LCD 2004 I2C` (or `20x4 LCD`). I2C backpack makes it a 4-pin part: VCC, GND, SDA, SCL.
DS1307 RTC	1	Search `DS1307`. I2C real-time clock (the firmware falls back to the sim clock if this is missing).
Buzzer	1	Search `buzzer` (piezo, 2 pins: + and –).
LED	7	Any LEDs. Recommended colors in order: red, orange, yellow, green, blue, purple, red.
VL53L0X (time-of-flight distance sensor)	7	Search `VL53L0X`. 4-pin breakout: VIN, GND, SDA, SCL, plus an XSHUT pin.

Part list at a glance

```
1x ESP32-S3 Dev Board
1x DHT22                      temperature & humidity
1x LCD 20x4 (I2C backpack)    display
1x DS1307 RTC                 real-time clock
1x Buzzer                     audio alarm
7x LED                        bay indicator lights
7x VL53L0X                    distance sensors (medicine bays)
-----------------------------------------------
22 parts total
```

4. ESP32-S3 pin assignments (functional map)

The table is the single source of truth for every wire in this build. It is copied exactly from the production firmware's pinout (`firmware/CareCube_ESP32_Firmware_v2`).

Purpose	GPIO / Power pin	Connects to
I2C — SDA (data)	GPIO21	LCD SDA, DS1307 SDA, VL53L0X #1–7 SDA
I2C — SCL (clock)	GPIO22	LCD SCL, DS1307 SCL, VL53L0X #1–7 SCL
DHT22 data	GPIO16	DHT22 DATA pin
Buzzer	GPIO2	Buzzer +
LED 1 — red	GPIO4	LED 1 anode
LED 2 — orange	GPIO12	LED 2 anode
LED 3 — yellow	GPIO19	LED 3 anode
LED 4 — green	GPIO18	LED 4 anode
LED 5 — blue	GPIO23	LED 5 anode
LED 6 — purple	GPIO13	LED 6 anode
LED 7 — red (emergency)	GPIO15	LED 7 anode
VL53L0X #1 XSHUT	GPIO25	Sensor #1 XSHUT
VL53L0X #2 XSHUT	GPIO26	Sensor #2 XSHUT
VL53L0X #3 XSHUT	GPIO27	Sensor #3 XSHUT
VL53L0X #4 XSHUT	GPIO32	Sensor #4 XSHUT
VL53L0X #5 XSHUT	GPIO33	Sensor #5 XSHUT
VL53L0X #6 XSHUT	GPIO14	Sensor #6 XSHUT
VL53L0X #7 XSHUT (emergency)	GPIO5	Sensor #7 XSHUT
Power	3V3	DHT22 VCC, LCD VCC, DS1307 VCC, all 7× VL53L0X VIN
Ground	GND	DHT22 GND, LCD GND, DS1307 GND, buzzer –, all LED cathodes, all VL53L0X GND

Wiring as a picture

Everything shares three groups of nets. Mentally split the build into these three groups and the whole circuit becomes simple:

```
GROUP A: POWER (3V3 + GND)
┌──────────────┐
│   3V3 net    │──── DHT22 VCC, LCD VCC, DS1307 VCC, VL53L0X #1..#7 VIN
│   GND net    │──── DHT22 GND, LCD GND, DS1307 GND, buzzer(-),
│              │     LED #1..#7 cathode, VL53L0X #1..#7 GND
└──────────────┘

GROUP B: I2C BUS (two wires everywhere)
GPIO21 ── SDA ─┬── LCD SDA
               ├── DS1307 SDA
               └── VL53L0X #1..#7 SDA        (9 devices, all
GPIO22 ── SCL ─┬── LCD SCL                     share the same
               ├── DS1307 SCL                  two wires)
               └── VL53L0X #1..#7 SCL

GROUP C: OUTPUTS (one wire each from the ESP32-S3)
GPIO16 ── DHT22 DATA
GPIO 2 ── Buzzer +
GPIO 4 ── LED 1 anode ... GPIO 15 ── LED 7 anode
GPIO25 ── VL53L0X #1 XSHUT ... GPIO5 ── VL53L0X #7 XSHUT
```

> **Why seven VL53L0X sensors share two wires?** All distance sensors are I2C devices that start with the same address (0x29). The firmware uses the **XSHUT** pin to wake them one at a time, giving each its own address (0x30–0x36) during startup. That is why every sensor gets its own XSHUT wire while SDA/SCL are shared. In simulation the sensors are optional; if one reports "NOT FOUND" the firmware simply skips it.

5. Recommended build order

Building in this order avoids re-touching parts and makes mistakes easy to spot:

1. **Place all 22 parts** on the canvas and spread them out.
2. Wire the **power rails** first (3V3 and GND) to every part that needs power.
3. Wire the **I2C bus** (SDA to GPIO21, SCL to GPIO22) for the LCD, RTC and all sensors.
4. Wire the **single-pin outputs**: DHT22 data, buzzer, the seven LEDs.
5. Wire each VL53L0X **XSHUT** to its dedicated GPIO.

6. Detailed wiring, step by step

6.1 Power — 3V3 net

Connect **3V3** on the ESP32-S3 to each of these pins:

From (ESP32-S3)	To (part pin)	Wire color suggestion
3V3	DHT22 **VCC**	red
3V3	LCD 20x4 **VCC**	red
3V3	DS1307 **VCC** (or 5V)	red
3V3	VL53L0X **#1 VIN**	red
3V3	VL53L0X **#2 VIN**	red
3V3	VL53L0X **#3 VIN**	red
3V3	VL53L0X **#4 VIN**	red
3V3	VL53L0X **#5 VIN**	red
3V3	VL53L0X **#6 VIN**	red
3V3	VL53L0X **#7 VIN**	red

6.2 Ground — GND net

Connect **GND** on the ESP32-S3 to each of these pins:

From (ESP32-S3)	To (part pin)	Wire color suggestion
GND	DHT22 **GND**	black
GND	LCD 20x4 **GND**	black
GND	DS1307 **GND**	black
GND	Buzzer **–**	black
GND	LED **#1–#7 cathode** (7 wires)	black
GND	VL53L0X **#1–#7 GND** (7 wires)	black

6.3 I2C bus — SDA (GPIO21) and SCL (GPIO22)

One wire from GPIO21 to every device's **SDA**; one wire from GPIO22 to every device's **SCL**. Nine devices total.

ESP32-S3	To (SDA)	ESP32-S3	To (SCL)
GPIO21	LCD **SDA**	GPIO22	LCD **SCL**
GPIO21	DS1307 **SDA**	GPIO22	DS1307 **SCL**
GPIO21	VL53L0X #1 **SDA**	GPIO22	VL53L0X #1 **SCL**
GPIO21	VL53L0X #2 **SDA**	GPIO22	VL53L0X #2 **SCL**
GPIO21	VL53L0X #3 **SDA**	GPIO22	VL53L0X #3 **SCL**

GPIO21	VL53L0X #4 **SDA**	GPIO22	VL53L0X #4 **SCL**
GPIO21	VL53L0X #5 **SDA**	GPIO22	VL53L0X #5 **SCL**
GPIO21	VL53L0X #6 **SDA**	GPIO22	VL53L0X #6 **SCL**
GPIO21	VL53L0X #7 **SDA**	GPIO22	VL53L0X #7 **SCL**

On real hardware the LCD/DS1307 breakout boards provide the I2C pull-up resistors; in the simulator this is handled automatically.

6.4 DHT22

DHT22 pin	ESP32-S3 pin	Wire color suggestion
DATA	**GPIO16**	purple / green
VCC	3V3	red
GND	GND	black

6.5 Buzzer

Buzzer pin	ESP32-S3 pin	Wire color suggestion
+	**GPIO2**	orange
–	GND	black

6.6 Seven LEDs (bay indicators)

Each LED's **anode** goes to its GPIO, **cathode** to GND.

LED	Color	Anode → GPIO	Cathode →
LED 1	red	GPIO4	GND
LED 2	orange	GPIO12	GND
LED 3	yellow	GPIO19	GND
LED 4	green	GPIO18	GND
LED 5	blue	GPIO23	GND
LED 6	purple	GPIO13	GND
LED 7	red	GPIO15	GND

6.7 Seven VL53L0X distance sensors (medicine bays 1–7)

Each sensor has the same 3 shared connections (VIN, GND, SDA, SCL) plus its own XSHUT wire. Only XSHUT differs per sensor.

Sensor	VIN	GND	SDA	SCL	XSHUT
#1 — slot A	3V3	GND	GPIO21	GPIO22	**GPIO25**
#2 — slot B	3V3	GND	GPIO21	GPIO22	**GPIO26**
#3 — slot C	3V3	GND	GPIO21	GPIO22	**GPIO27**
#4 — slot D	3V3	GND	GPIO21	GPIO22	**GPIO32**
#5 — slot E	3V3	GND	GPIO21	GPIO22	**GPIO33**
#6 — slot F	3V3	GND	GPIO21	GPIO22	**GPIO14**
#7 — emergency	3V3	GND	GPIO21	GPIO22	**GPIO5**

7. Wiring checklist

Tick each box as you complete it. This is the fastest way to catch a missed wire before you ever run the simulation.

Power

- ☐ 3V3 → DHT22 VCC
- ☐ 3V3 → LCD VCC
- ☐ 3V3 → DS1307 VCC
- ☐ 3V3 → VL53L0X #1–#7 VIN (7 wires)
- ☐ GND → DHT22 GND
- ☐ GND → LCD GND
- ☐ GND → DS1307 GND
- ☐ GND → Buzzer –
- ☐ GND → LED #1–#7 cathode (7 wires)
- ☐ GND → VL53L0X #1–#7 GND (7 wires)

I2C bus

- ☐ GPIO21 → LCD SDA
- ☐ GPIO22 → LCD SCL
- ☐ GPIO21 → DS1307 SDA
- ☐ GPIO22 → DS1307 SCL
- ☐ GPIO21 → VL53L0X #1–#7 SDA (7 wires)
- ☐ GPIO22 → VL53L0X #1–#7 SCL (7 wires)

Signals

- ☐ GPIO16 → DHT22 DATA
- ☐ GPIO2 → Buzzer +
- ☐ GPIO4/12/19/18/23/13/15 → LED #1–#7 anode
- ☐ GPIO25/26/27/32/33/14/5 → VL53L0X #1–#7 XSHUT

Do a final visual review

- Any wire that should hit **GND** but doesn't will usually be the difference between "works" and "nothing happens" — double-check the black wires.
- Confirm no SDA wire accidentally touches SCL (crossed wires are the #1 wiring error in this build).
- Confirm each LED's anode/cathode orientation; an inverted LED stays dark.

8. Load the code

1. In the Cirkit workspace click the **Code** tab (top-right area). It opens an Arduino sketch editor already targeted at the ESP32-S3.
2. Delete the default sketch.
3. Open `cirkit/sketch.ino` from this repository and paste **all of it** into the Code tab.
4. Save (the project auto-saves to the cloud).

The sketch already handles the simulator specifics:

- Connects to the virtual WiFi network `CirkitWifi` (no password).
- Uses `WiFiClientSecure` + `setInsecure()` to talk TLS to the HiveMQ Cloud broker on port 8883.
- Publishes to `carecube/carecube-001/status` every 30 seconds.
- Subscribes to schedule changes and alarm-stop commands.
- Initializes every VL53L0X gracefully (missing ones are skipped, never a crash loop) and falls back to the simulation clock if the DS1307 is missing.

> The MQTT client id in the sketch is `carecube-001-cirkit`. Only one client may use that id on the broker at a time. If your Wokwi demo is also running, stop it (or change the id suffix) or you will see connection churn. See Chapter 11.

9. Run the simulation — what to expect

Click **Run** / **Start simulation** in the Simulate area.

Serial console (bottom panel)

You should see, in order:

```
Connecting to WiFi: CirkitWifi
.WiFi connected!
IP: 172.20.0.xx
Connecting to MQTT... connected!
Subscribed to: carecube/carecube-001/schedule/set
Subscribed to: carecube/carecube-001/alarm/stop
...
==============================
      CARE CUBE READY
==============================
Time: 19:12:05 | T:24.3C H:51.2% | MQTT:Y | S1:0 S2:0 S3:0 S4:0 S5:0 S6:0 S7:0
...
MQTT -> [carecube/carecube-001/status]
{"device_id":"carecube-001","temperature":24.3,"humidity":51.2,"timestamp":1736090930}
```

The `S1:0 ... S7:0` values stay at a small number in the simulator if the distance sensors are not interactive — that is normal and harmless for this build.

LCD display

```
Line 1:  19:12:05    W:Y M:Y
Line 2:  13/09/2026
Line 3:  T:24.3C H:51.2%
Line 4:  Next: 08:00PM
```

Interactive parts

- Drag the **DHT22** value in the simulator (temperature/humidity slider). Within ~30 seconds the new values appear in the console and on the LCD.
- At each dose time the matching **LED** lights and the **buzzer** sounds; the console prints `REMINDER -> SENSOR n TAKE MEDICINE`.

10. Verify end-to-end with the Care Cube app

1. Build and install the app on your phone (the usual `flutter build apk --debug` + adb workflow), leaving the Cirkit simulation running in the browser.
2. Open the app and sign in.
3. Go to the **Home** tab.
4. The **connection status card** should change to "connected".

5. The **Temperature** and **Humidity** cards update every ~30 seconds, and immediately when you move the DHT22 slider in Cirkit.
6. Change a dose time in the **Medicine** tab and save — the app publishes `carecube/carecube-001/schedule/set` and Cirkit's serial console prints `Schedule updated via MQTT` with the new time.

11. Troubleshooting

Symptom	Check
WiFi never connects	Make sure the sketch connects to `CirkitWifi` exactly (not `Wokwi-GUEST`). The simulator only supports this open access point.
MQTT fails with `failed, rc=...`	HiveMQ Cloud needs TLS on port 8883; the sketch already calls `setInsecure()`. Verify the broker host, username and password in the sketch match `lib/config/app_config.dart` in the app. As a connectivity sanity check, open Cirkit's official "MQTT Temperature Monitor" example first — if that also fails, the issue is general internet/TLS from the simulator.
Messages arrive, then stop	Another client is using the same MQTT client id on the broker (e.g. the Wokwi demo). Stop the other simulation, or append a different suffix to `MQTT_CLIENT_ID` in the sketch.
VL53L0X prints `NOT FOUND - skipping`	Normal in the simulator. Temperature/humidity streaming and schedule updates still work. Distance sensing is only used to detect "medicine taken" and emergency doses.
RTC prints `NOT FOUND`	The sketch falls back to the simulation clock, so dose timers still fire on time.
App shows "disconnected"	App and firmware must use the same broker, device id (`carecube-001`) and topics. Confirm on the Home tab that the connection card goes green, and that the Cirkit console shows `MQTT:Y` in the status line.
LEDs stay dark at a dose time	Confirm dose is enabled (all 6 slots default ON) and check the LED's anode/cathode wiring. The simulator runs faster than wall-clock; wait for the next scheduled minute to elapse.
Nothing appears on the LCD	Check I2C wiring (SDA → GPIO21, SCL → GPIO22) and that the LCD part is the **I2C** variant. If RTC is also missing, firmware still runs but you lose the display.

Appendix A — Complete net list

Every wire in the build, as a flat list.

3V3 net

```
3V3  -> DHT22 VCC
3V3  -> LCD VCC
3V3  -> DS1307 VCC
3V3  -> VL53L0X #1 VIN, #2 VIN, #3 VIN, #4 VIN, #5 VIN, #6 VIN, #7 VIN
```

GND net

```
GND  -> DHT22 GND
GND  -> LCD GND
GND  -> DS1307 GND
GND  -> Buzzer -
GND  -> LED #1 cathode .. LED #7 cathode
GND  -> VL53L0X #1 GND .. VL53L0X #7 GND
```

SDA net (GPIO21)

```
GPIO21 -> LCD SDA
GPIO21 -> DS1307 SDA
GPIO21 -> VL53L0X #1..#7 SDA
```

SCL net (GPIO22)

```
GPIO22 -> LCD SCL
GPIO22 -> DS1307 SCL
GPIO22 -> VL53L0X #1..#7 SCL
```

Individual signals

```
GPIO16 -> DHT22 DATA
GPIO2  -> Buzzer +
GPIO4  -> LED 1 anode     (red)
GPIO12 -> LED 2 anode     (orange)
GPIO19 -> LED 3 anode     (yellow)
GPIO18 -> LED 4 anode     (green)
GPIO23 -> LED 5 anode     (blue)
GPIO13 -> LED 6 anode     (purple)
GPIO15 -> LED 7 anode     (red, emergency)
GPIO25 -> VL53L0X #1 XSHUT
GPIO26 -> VL53L0X #2 XSHUT
GPIO27 -> VL53L0X #3 XSHUT
GPIO32 -> VL53L0X #4 XSHUT
GPIO33 -> VL53L0X #5 XSHUT
```

```
GPIO14 -> VL53L0X #6 XSHUT
GPIO5  -> VL53L0X #7 XSHUT
```

Appendix B — What the firmware does

Quick map of the state machine so simulation behavior makes sense:

State	Behavior	Published event
MAIN_SCREEN	Shows time, date, temp/humidity and next dose on the LCD every 0.5 s.	—
MEDICINE_REMINDER (dose hour/minute reached)	LED of that bay lights, buzzer sounds. If the bay's distance sensor reads > 90 mm for 3 consecutive readings, the dose is logged as taken.	reminder_started, then taken
MEDICINE_TAKEN	Confirmation screen shown for 30 s, LED stays on.	—
EMERGENCY_TAKEN (sensor #7 edge detected)	Buzzer + LED 7 for 60 s.	emergency_taken

"caregiver_stopped" is published when the app sends the remote alarm-stop command on `carecube/carecube-001/alarm/stop`.

MQTT topic map

```
App -> Firmware   carecube/carecube-001/schedule/set     (JSON array, 6 slots)
App -> Firmware   carecube/carecube-001/alarm/stop       (caregiver only)
Firmware -> App   carecube/carecube-001/status           {device_id,temperature,humidity,timestamp}
Firmware -> App   carecube/carecube-001/event            {device_id,type,sensor_number,timestamp}
```

> Repo files referenced by this guide: `cirkit/sketch.ino`, `wokwi/sketch.ino` (original simulator port), `firmware/CareCube_ESP32_Firmware_v2/CareCube_ESP32_Firmware_v2.ino` (production firmware this build mirrors), `lib/config/app_config.dart` and `lib/services/mqtt_service.dart` (app-side MQTT contract).